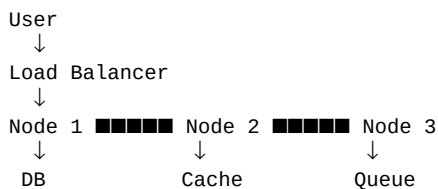


Distributed Systems — CAP, Consistency & Distributed Transactions

Hinglish • CAP • Consistency • Quorum • 2PC • Saga • Locks • Failure Handling • Interview Prep

1. Distributed System Kya Hai?

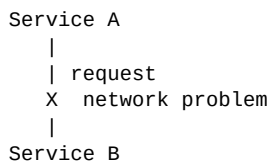
Distributed system mein multiple machines/services milkar ek logical system provide karte hain. User ko ideally ek application dikhta hai, but backend mein multiple nodes communicate kar rahe hote hain.



2. Single Machine vs Distributed

Single machine	Distributed system
Local communication fast	Network communication
One failure domain	Multiple failure domains
Simple consistency	Distributed consistency problems
Limited scaling	Horizontal scaling possible

3. The Fundamental Reality: Network Can Fail



Distributed system mein timeout, packet loss, delayed response, duplicate message, partial failure aur node crash normal possibilities hain. Isliye design ko failure ko assume karke banana chahiye.

4. Partial Failure

Single application mein process crash hua to obvious failure dikhta hai. Distributed system mein ek service healthy ho sakti hai aur doosri unreachable. Ye partial failure hai.

Order Service → healthy
Payment Service → unreachable

Overall workflow → uncertain

5. CAP Theorem

CAP theorem distributed data system ke context mein kehta hai ki network partition ke time system simultaneously strong Consistency aur full Availability dono guarantee nahi kar sakta. Partition tolerance ko distributed network mein ignore nahi kiya ja sakta.



During a network partition:
choose consistency or availability trade-off

6. C — Consistency

CAP context mein consistency ka matlab hai successful read ko latest committed value ke saath consistent result dena. Simplified mental model: system ke nodes conflicting answers na dein.

Write:

balance = 100

Read:

Node A → 100

Node B → 100

7. A — Availability

Availability ka matlab request ko system se response milta rahe, even when some nodes are unavailable. CAP mein availability ka exact meaning request-level success hai, not necessarily latest data.

8. P — Partition Tolerance

Network partition ka matlab nodes ke beech communication break/delay ho sakta hai.

Node A X Node B

Both nodes alive,
but cannot communicate.

Real distributed networks mein partitions possible hain, so practical design ko P tolerate karna padta hai.

9. CP vs AP

CP	AP
Partition ke time consistency prefer	Partition ke time availability prefer
Some requests reject/wait	Requests may return available/stale state
Strong coordination	Eventual convergence common
Example use: critical coordinated state	Example use: highly available distributed data

10. Practical CAP Example

Two replicas:

A = balance 100
B = balance 100

Network partition:
A X B

Client writes:
A → balance 80

Now B still has 100.

If B must immediately serve a consistent value:
availability may be sacrificed.

If B keeps serving:
availability maintained,
but stale/conflicting data may be observed.

CAP ka real point ye trade-off hai, not simply 'system is C, A or P' as a permanent label.

11. Strong Consistency

Successful write ke baad subsequent reads latest value observe karne ki stronger guarantee de sakte hain.

```
WRITE 100 → 80
  ↓
READ
  ↓
80
```

12. Eventual Consistency

Temporary stale reads allowed hote hain, but if updates stop and system recovers, replicas eventually converge.

t0: A=100 B=100
t1: A=80 B=100
t2: replication
t3: A=80 B=80

13. Read-After-Write Consistency

User ne profile update ki aur immediately GET kiya. User expect karta hai ki usko apna latest update dikhe.
Read-after-write consistency isi user experience ko address karti hai.

POST /profile
→ name = Sonu

Immediately:
GET /profile
→ name = Sonu

Read replica lag ho to immediate GET stale replica se aa sakta hai. Sensitive flows mein primary/read-your-write routing useful ho sakti hai.

14. Monotonic Reads

A user ko pehle newer value dikhne ke baad later older value nahi dikhni chahiye. Session-aware routing/versions jaisi techniques help kar sakti hain.

15. Quorum

Replicated system mein read/write operations multiple replicas ke saath coordinate kiye ja sakte hain. Simplified quorum model mein N replicas, W write acknowledgements aur R read acknowledgements consider karte hain.

$N = 3$ replicas

$W = 2$

$R = 2$

If $R + W > N$

→ overlap is guaranteed

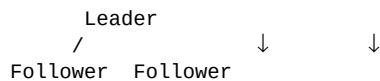
3 replicas:

$R=2, W=2$

$2+2 > 3$

Actual consistency guarantees system implementation, failure model and read/write semantics par depend karte hain; quorum formula ko blindly universal guarantee mat samjho.

16. Leader / Follower



Writes → Leader

Reads → followers / leader depending on consistency

Leader-based replication coordination simplify kar sakti hai. Leader failure par election/failover required hota hai.

17. Split Brain

Split brain situation mein multiple nodes simultaneously leader/authority behave kar sakte hain, causing conflicting writes.

Network partition

Group A → thinks A is leader

Group B → thinks B is leader

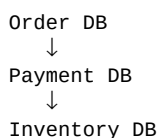
Both accept writes

→ conflict risk

Quorum, fencing, leader election aur coordination mechanisms split-brain risk control karte hain.

18. Distributed Transaction

Multiple services/databases ke across business operation ko atomic banane ka problem distributed transaction hai.



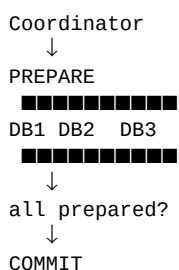
Need:

all succeed?

or compensate?

19. Two-Phase Commit (2PC)

2PC distributed transaction coordinator ke through participants se prepare aur commit phases mein agreement leta hai.

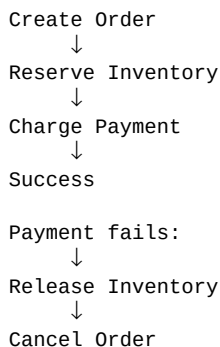


20. 2PC Problems

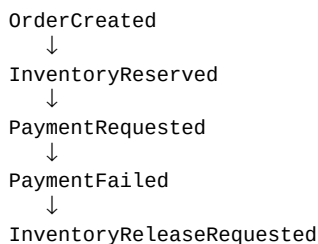
- Coordinator dependency.
- Lock/resource holding can be long.
- Network failures complicate progress.
- Operational complexity.
- Not a universal fit for microservice business workflows.

21. Saga Pattern

Saga distributed workflow ko local transactions mein split karta hai. Failure par compensating actions run hote hain.

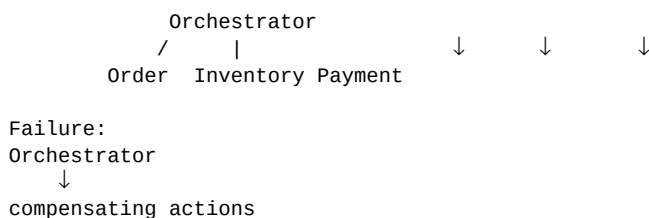


22. Saga Choreography



Services events consume karke next action trigger karti hain.

23. Saga Orchestration



Complex workflows mein central orchestration visibility improve kar sakti hai.

24. Idempotency

Network retry ke wajah se same command/event multiple times aa sakta hai. Idempotency ensure karti hai ki same logical operation repeated hone par duplicate side effect na ho.

```
idempotencyKey = abc123
```

```
Request 1:  
create payment → success
```

```
Request 2:  
same key  
→ return existing result  
→ don't charge again
```

25. Distributed Lock

Distributed lock ka purpose multiple workers/nodes ko same critical operation simultaneously execute karne se prevent karna hai.

```
Worker A ■■  
          ■→ lock(resource)  
Worker B ■■
```

```
A gets lock  
B waits/fails
```

```
A finishes  
→ release lock
```

Redis-based locks exist, but correctness requirements ke according lock lease, expiration, ownership token and failure behavior carefully design karo.

26. Lock Problem

Lock holder crash kar sakta hai. Network delay ke wajah se stale lock owner operation continue kar sakta hai. Fencing tokens jaisi techniques critical systems mein stale actors ko reject karne mein help kar sakti hain.

27. Race Condition

Two requests:

```
A reads balance = 100  
B reads balance = 100
```

```
A writes 80  
B writes 50
```

Expected business result may differ.

Need:
transaction / atomic update / locking / versioning

28. Optimistic Concurrency

```
row version = 10
```

```
UPDATE ... WHERE id=1 AND version=10
```

```
If 0 rows updated:  
someone changed it first  
→ conflict
```

Optimistic locking low-conflict workflows mein useful hai.

29. Pessimistic Locking

Database row ko transaction ke duration mein lock karke conflicting writes ko wait/block karaya ja sakta hai. Long transactions lock contention create kar sakti hain.

30. Compare-and-Set

```
UPDATE account  
SET balance = balance - 100  
WHERE id = 1  
    AND balance >= 100
```

Atomic database operation race conditions avoid karne ka simple powerful pattern ho sakta hai.

31. Exactly-Once Myth

Networked systems mein end-to-end exactly-once side effects ko casually assume nahi karna chahiye. Practical design usually at-least-once delivery + idempotent processing + unique constraints/state transitions par depend karta hai.

32. Duplicate Messages

```
Queue
↓
Message #123
↓
Consumer
↓
crash before ACK
↓
Message #123 redelivered

Consumer:
eventId already processed?
→ yes → skip
```

33. Retry Storm

```
Dependency slow
↓
Service retries
↓
more traffic
↓
dependency even slower
↓
more retries
```

Solution:
timeout
backoff
jitter
retry budget
circuit breaker

34. Backpressure

Producer consumers se faster messages create kare to queue/backlog grow karega. Backpressure ka goal producer rate ko consumer capacity ke according control karna hai.

Producer: 10,000 msg/s
Consumer: 2,000 msg/s

Backlog ↑

Need:
throttle
scale consumers
batching
load shedding
or queue capacity strategy

35. Clock Problem

Different machines ke clocks perfectly synchronized nahi hote. Distributed ordering ko wall-clock timestamp alone se determine karna dangerous ho sakta hai.

Node A time: 10:00:01.100
Node B time: 10:00:00.950

Which event happened first?
Timestamp alone may mislead.

Logical clocks, sequence numbers, database versions ya event ordering mechanisms use kiye ja sakte hain.

36. Logical Ordering

Distributed systems mein event sequence ko explicitly represent karna useful hota hai.

```
message 101  
message 102  
message 103
```

```
lastProcessed = 102  
→ process 103
```

37. Failure Detection

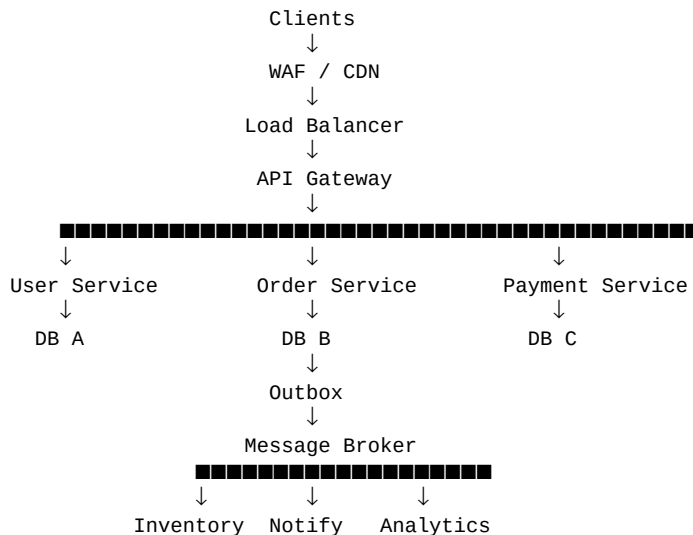
A node silent hai to iska matlab necessarily node dead nahi. Network delay aur crash ko distinguish karna difficult hai. Timeouts are practical failure detectors, but they are imperfect signals.

38. Network Partition Scenario

```
Node A                Node B  
|                      |  
|----- partition -----X  
|                      |  
accept?                accept?
```

Business requirement decides:
availability vs consistency trade-off

39. Production Architecture



Observability:
Metrics + Logs + Traces

40. Which Consistency Should You Choose?

Question ko 'CAP mein CP ya AP?' se start karne ke bajay business requirement se start karo.

- Is stale data acceptable?
- Can user retry safely?
- Can operation be compensated?
- What happens during partition?
- What is the cost of rejecting a request?
- Does the business require read-after-write?

41. Example: Social Feed

Social feed mein few seconds old data often acceptable ho sakta hai. Availability and scale important ho sakte hain, so eventual consistency can be acceptable.

42. Example: Payment

Payment state mein incorrect duplicate charge unacceptable hai. Idempotency, strong state transitions, reconciliation and careful failure handling important hain.

43. Example: Inventory

Inventory reservation overselling prevent karne ke liye atomic updates/locking/version checks important ho sakte hain. Exact architecture traffic and business requirements par depend karega.

44. Example: Notification

Notification delivery usually asynchronous ho sakti hai. Temporary queue lag acceptable ho sakta hai, provided retries, DLQ and delivery status tracking exist karein.

45. Observability Metrics

- Request rate.

- p50/p95/p99 latency.
- Timeout rate.
- Retry count.
- Circuit breaker opens.
- Queue depth / consumer lag.
- Replication lag.
- Leader changes.
- Lock contention.
- Transaction conflicts.
- Saga failures/compensations.
- Reconciliation mismatches.

46. Local Experiments

Node.js + Redis + PostgreSQL/MySQL + RabbitMQ/Kafka ke saath practical experiments:

- Run 3 service instances and introduce network delay.
- Simulate duplicate queue messages.
- Kill a consumer before ACK.
- Implement idempotency table.
- Implement optimistic locking.
- Create Order → Payment Saga.
- Make Payment fail and trigger compensation.
- Implement Outbox table + publisher.
- Add retry with exponential backoff+jitter.
- Add circuit breaker.
- Observe everything in Prometheus + Grafana.

47. Interview Q&A;

Question	Short answer
Distributed system kya?	Multiple machines/services collaborating over a network.
CAP?	During partition, consistency and availability trade off.
C?	Consistent reads according to the chosen consistency guarantee.
A?	Requests continue receiving responses.
P?	System tolerates network partition.
CP?	Partition ke time consistency prefer.
AP?	Partition ke time availability prefer.
Eventual consistency?	Replicas may temporarily differ but converge after updates settle.
Quorum?	Coordinate reads/writes across a replica set; exact guarantees depend on system semantics.
2PC?	Coordinator-based prepare then commit protocol.
Saga?	Distributed workflow using local transactions and compensations.
Outbox?	Persist event intent with DB transaction, publish asynchronously.
Idempotency?	Repeated logical request does not duplicate side effects.
Split brain?	Multiple nodes act as authority simultaneously.
Distributed lock?	Coordinate access to a shared critical resource across nodes.

48. 80/20 Mental Model

DISTRIBUTED SYSTEM

Machines



Network



Partial failures are normal

Think about:

- consistency
- availability
- partition

- replication
- quorum
- leader election
- transactions
- Saga
- Outbox
- idempotency
- locks
- retries
- timeouts
- circuit breakers
- observability

Golden interview line:

"In a distributed system I assume the network can fail, messages can be delayed or duplicated, and nodes can crash. I first define the business consistency requirement, then choose replication and coordination accordingly. For cross-service workflows I prefer local transactions with Saga/compensation, reliable event publication through Outbox, and idempotent consumers with bounded retries, timeouts and circuit breakers."

49. Quick Revision Diagram

